

Git Branching & Merging — The Real Mental Model + Exact Commands

The mental model

Start with the commit, because everything else is built on it. A **commit is a snapshot** of your entire project at a moment, plus a pointer to its parent commit (the one before it). Chain those parent-pointers together and you get history — not a straight line necessarily, but a graph that only ever grows backward in time. A commit never changes; it's identified by a SHA-1 hash of its contents. If a commit's content changes, its hash changes, so it's effectively a *different* commit. Hold onto that — it's why rebase behaves the way it does.

Now the thing beginners get wrong: **a branch is not a copy of your files, and it's not a container**. A branch is a tiny movable label — literally a 41-byte file at `.git/refs/heads/<name>` containing one commit hash. That's the whole branch. `main` and `feature` are both just sticky notes stuck onto particular commits. This is why creating a branch is instant and free even in a giant repo — Git writes one small file. "Deleting a branch" just peels off the sticky note; the commits underneath may live on.

There's one more pointer that ties it together: `HEAD`, the "you are here" arrow. Normally `HEAD` doesn't point at a commit directly — it points at a *branch* (e.g. `HEAD → main`), and that branch points at a commit. The magic is what happens when you commit: Git makes the new commit, then **slides the branch label that HEAD is pointing at forward** to the new commit. The branch follows you automatically. That's the entire trick of how branches "grow."

Merging is reconciling two labels back together, and there are exactly two situations. If the branch you're merging *in* is simply further down the same road with no forks — `main` hasn't moved since you branched — Git does a **fast-forward**: it just slides `main`'s label up to catch up. No new commit, nothing to reconcile. But if *both* branches have new commits (the history forked), Git does a **three-way merge**: it finds the common ancestor (the "merge base"), compares both sides against it, and creates a brand-new **merge commit that has two parents**, tying the fork back together. Analogy: two cooks photocopied the same recipe and each scribbled edits; to combine, you compare both copies against the original and produce one reconciled recipe.

Rebase is the alternative to a three-way merge, and it's a different philosophy. Instead of making a merge commit, rebase *rewrites* your commits as if you'd started from the tip of `main` all along — it replays your changes one by one onto the latest `main`, producing new commits with new hashes and a clean straight line. The payoff is linear, readable history. The price is that you've rewritten history (new hashes), which is fine for your private work and **dangerous for anything you've already shared** — more on that in Traps.

What you need first

- **A commit** — a saved snapshot + message; the atomic unit of history (`git commit`).
- **The working directory / staging area** — your files, and the "on-deck" set you've `git add` -ed for the next commit.
- **HEAD** — the pointer to your current branch (or, when "detached," directly to a commit).
- **A remote** (e.g. `origin`) — a copy of the repo elsewhere (GitHub/GitLab); `main` and `origin/main` are *different* labels.
- **Git 2.23+** — needed for `git switch` / `git restore` ; check with `git --version` . (Released 2019, so almost certainly fine.)
- Basic terminal comfort — `cd` into a repo, run commands, read output.

The steps

A complete feature workflow, start to finish.

0. One-time setup so new repos default to `main` :

```
git config --global init.defaultBranch main
git --version    # confirm you're on 2.23+ for switch/restore
```

1. See where you are. The graph is the source of truth:

```
git status          # current branch + working-tree state
git branch          # list local branches; * marks current
git log --oneline --graph --all  # visualize the whole commit DAG
```

2. Create a branch and switch to it (modern command):

```
git switch -c feature-login    # create 'feature-login' and move HEAD onto
it
# older equivalent, still valid:
# git checkout -b feature-login
```

3. Do work and commit. Each commit slides the `feature-login` label forward:

```
git add .
git commit -m "Add login form"
# ...more edits...
git add .
git commit -m "Validate credentials"
```

4. **Switch back to main and update it** so it reflects the latest shared work:

```
git switch main
git pull                # fast-forward main to match origin/main
```

5a. **Merge — the default, history-preserving path.** Merge the feature *into* main (you must be *on* the receiving branch):

```
git switch main
git merge feature-login
```

- If main hasn't moved, this **fast-forwards** (no merge commit).
- If both diverged, Git makes a **merge commit** and opens your editor for its message.
- To *force* a merge commit even when a fast-forward is possible (keeps the branch's existence visible in history):

```
git merge --no-ff feature-login
```

5b. **Rebase — the alternative, linear-history path.** Replay your feature commits on top of the latest main *before* merging:

```
git switch feature-login
git rebase main          # replay feature's commits onto main's tip
git switch main
git merge feature-login   # now a clean fast-forward
```

6. **If a merge or rebase hits a conflict**, Git pauses and marks the files. Resolve them:

```
git status                # lists "Unmerged paths" = files to fix
# open each conflicted file; find and edit the markers:
# <<<<<<< HEAD
# your side (current branch)
# =====
# their side (incoming branch)
# >>>>>>> feature-login
# delete the markers, keep the correct combined result, save.
git add <resolved-file>    # mark each file resolved
git merge --continue       # finish a merge (or: git commit)
# during a rebase instead:
# git rebase --continue
```

Bail out cleanly if you'd rather not finish:

```
git merge --abort          # restore to exactly before the merge
git rebase --abort         # same, for a rebase
```

7. Push and clean up:

```
git push -u origin main    # push main; -u sets upstream (first time)
git branch -d feature-login # delete merged branch (safe: refuses if
                             unmerged)
git push origin --delete feature-login # delete it on the remote too
```

8. Undo button if a merge/rebase went wrong (do this *before* new risky operations):

```
git reset --hard ORIG_HEAD    # ORIG_HEAD = where HEAD was before the
merge/rebase
# gone too far? find any prior state and jump to it:
git reflog                   # journal of everywhere HEAD has been
git reset --hard HEAD@{2}    # reset to a specific reflog entry
```

Traps

- **Thinking a branch copies your files.** It doesn't — it's one pointer. Switching branches doesn't "save" your current branch's files somewhere; the commits already hold them. Untracked/uncommitted changes ride along and can block a switch — commit or `git stash` them first.
- **Merging in the wrong direction.** `git merge X` brings X *into your current branch*. To update main from a feature, you must `git switch main` **first**. Beginners often merge main into their feature and wonder why main didn't change.
- **The golden rule of rebase: never rebase commits you've already pushed/shared.** Rebase creates new hashes; if teammates based work on the old commits, you've forked history and created a mess. Rebase only local, unpushed work. (Merge is always safe here.)
- **`git pull` surprises.** A plain `pull` is `fetch` + `merge` and can create noisy merge commits or unexpected conflicts. Prefer `git pull --ff-only` (fail loudly instead of auto-merging) or set `git config --global pull.rebase true` if your team wants linear history. This is a genuine team-preference choice, not a universal right answer.
- **Panicking at conflict markers.** `<<<<<<<`, `=====`, `>>>>>>>` are not corruption — they're Git showing you both versions. You edit the file to the final desired state, remove *all three* markers, then `git add`. Forgetting to delete a marker line leaves literal `=====` in your code.
- **`git reset --hard` on work you care about.** It discards uncommitted changes permanently. Only use it when you mean it; reach for `git stash` if you might want the changes back.
- **`-d` vs `-D` when deleting.** `git branch -d` refuses to delete an unmerged branch (a safety net). `git branch -D` force-deletes — you can genuinely lose commits this way if they aren't on

another branch.

- **"Detached HEAD" alarm.** If you `git switch --detach <hash>` or check out a raw commit, HEAD points at a commit, not a branch — new commits there belong to *no branch* and can be garbage-collected. To keep work, make a branch: `git switch -c new-branch`.
- **Assuming lost commits are gone forever.** They usually aren't. `git reflog` remembers where HEAD has been (default ~90 days). Most "I destroyed everything" situations are a `git reset --hard HEAD@{n}` away.

Cheat sheet

```
# --- inspect ---
git status                # current branch + working state
git branch                # list local branches (* = current)
git branch -a             # include remote-tracking branches
git log --oneline --graph --all  # visualize the commit graph
git --version             # need >= 2.23 for switch/restore

# --- create / switch (modern) ---
git switch <branch>       # move to existing branch
git switch -c <branch>    # create + switch
git switch -              # switch to previous branch
# old equivalents: git checkout <b> / git checkout -b <b>

# --- rename / delete ---
git branch -m <new-name>  # rename current branch
git branch -d <branch>    # delete (safe; refuses if unmerged)
git branch -D <branch>    # force delete
git push origin --delete <branch> # delete on remote

# --- merge ---
git switch main           # be ON the receiving branch first
git merge <branch>        # fast-forward OR three-way merge
git merge --no-ff <branch> # force a merge commit
git merge --abort         # cancel an in-progress merge
git merge --continue      # finish after resolving conflicts

# --- rebase (linear history; NEVER on shared commits) ---
git switch <feature>
git rebase main           # replay feature onto main's tip
git rebase -i main        # interactive: squash/reorder/edit
git rebase --continue | --skip | --abort

# --- conflicts ---
# edit files, remove <<<<<< ===== >>>>>> markers, then:
git add <file>            # mark resolved
git merge --continue      # (or git rebase --continue)

# --- sync with remote ---
git pull --ff-only        # update without surprise merge commits
git push -u origin <branch> # push + set upstream (first push)

# --- undo / recover ---
```

```
git reset --hard ORIG_HEAD      # undo the last merge/rebase
git reflog                      # journal of past HEAD positions
git reset --hard HEAD@{n}       # jump back to a reflog entry
git stash / git stash pop       # shelve / restore uncommitted work

# --- mental model in one line ---
# commit = snapshot(+parent) · branch = movable label(41-byte file)
# HEAD = "you are here" · commit slides the active label forward
# fast-forward = slide label · 3-way merge = new 2-parent commit
# rebase = rewrite commits onto a new base (new hashes)
```

Sources: - [Git Internals — Git References \(git-scm.com\)](#) - [Git Refs and the Reflog \(Atlassian\)](#) - [git switch vs git checkout \(Refine\)](#) · [phoenixNAP](#) - [Merge vs Rebase vs Fast-Forward \(Medium\)](#) · [Merge vs Rebase \(Atlassian, via Refine\)](#) - [git-merge Documentation \(git-scm.com\)](#) · [Resolving merge conflicts \(Atlassian\)](#) - [git reflog recovery & ORIG_HEAD \(Icinga\)](#) · [Recovering lost commits with reflog \(Graphite\)](#)